

Git Workflow Cheatsheet

Dưới đây là một **Git Workflow Cheatsheet** được thiết kế mạch lạc, dễ hiểu, tập trung vào những thao tác cốt lõi nhất để bạn có thể chia sẻ trực tiếp với team mình.

0. Chuẩn bị môi trường

Trước khi thực sự bắt tay vào làm dự án, trước hết bạn phải đảm bảo rằng bạn đã có đủ các công cụ và môi trường để có thể bắt đầu.

☐

1. Cài đặt **Git**.

☐

2. Có tài khoản **Github** hoặc **Gitlab**.

☐

3. Tạo sẵn một thư mục trống mang tên dự án (Sau này sẽ làm việc nhiều ở thư mục này).

Một khi bạn đã hoàn thành 3 nhiệm vụ trên, ta có thể bắt tay vào với Git Workflow.

A. Tải dự án về máy (Chỉ làm 1 lần duy nhất)

Khi mới bắt đầu, trước tiên bạn cần phải có dự án trên máy mình trước khi có thể bắt đầu code. Việc khởi tạo file và cấu trúc dự án là trách nhiệm của **Team Lead**. Sau khi leader thông báo đã khởi tạo dự án, bạn cần "kéo" toàn bộ code từ kho chứa (Repository) trên mạng (GitHub, GitLab,...) về máy tính của mình.

1. Khởi tạo Git:

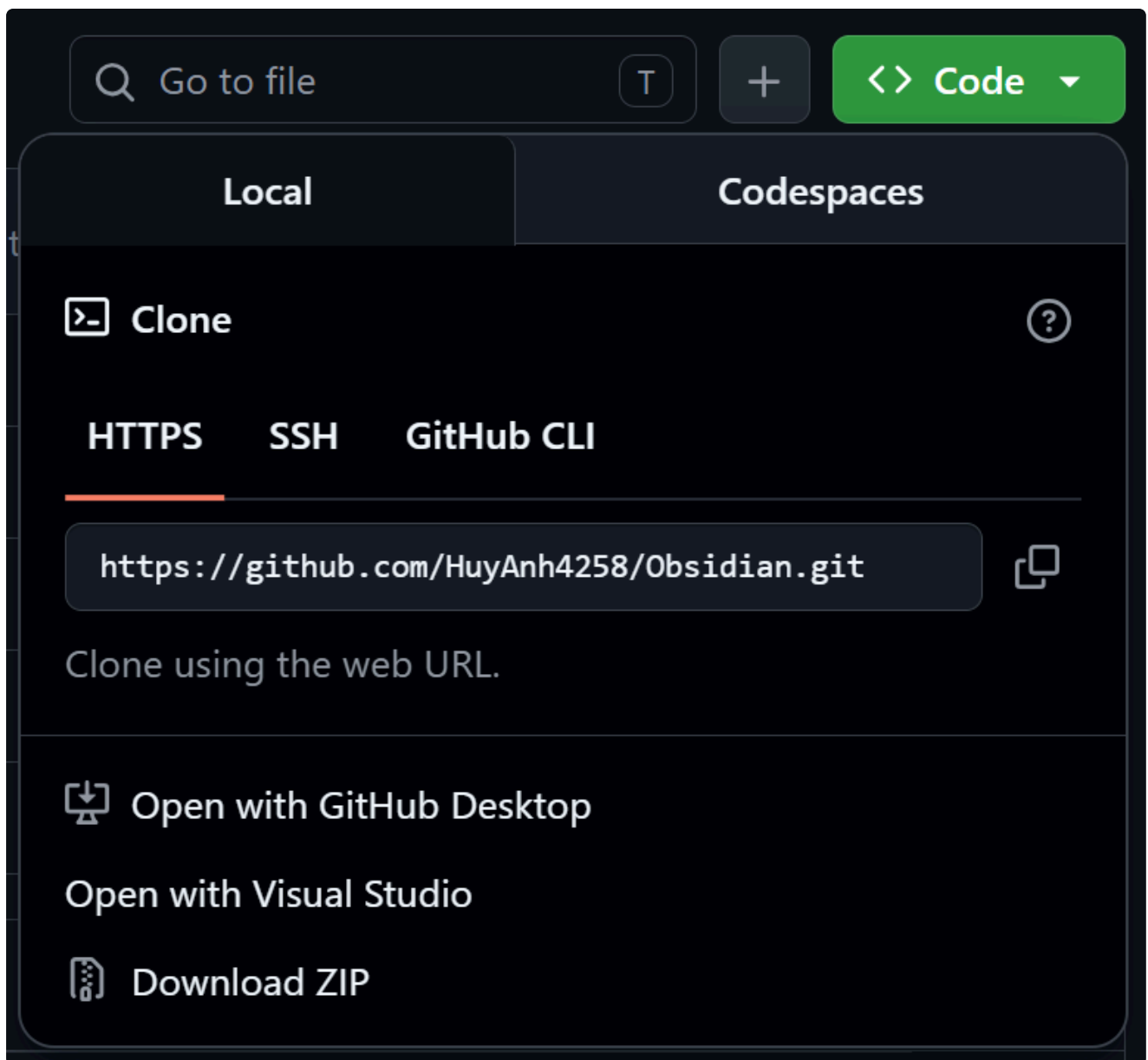
Sau khi đã tải Git và đăng nhập Github hoặc Gitlab như ở trên, bây giờ bạn cần phải cho Git biết rằng bạn đang muốn làm dự án ở đâu. Để khởi tạo Git ở local, hãy làm theo các bước sau:

- B1: Mở thư mục dự án trống mà bạn đã tạo.
- B2: Bên trong thư mục, ấn chuột phải vào chỗ trống.
- B3: Ở danh sách chọn, ấn "Open in Terminal" để mở terminal tại folder này.

Đây chính là nơi mà bạn sẽ luôn luôn gõ lệnh để đồng bộ code với team.

2. Kết nối đến repo và clone dự án:

Để clone một repo dự án về máy tính (local), trước hết ta phải có địa chỉ repo (URL). Với Github, đường dẫn của repo ở dạng: `https://github.com/<username>/<repo-name>.git`



Sau khi đã có đường dẫn url của repo, hãy mở terminal (Command Prompt) trên máy và chạy dòng lệnh sau:

```
git clone <đường-dẫn-url-của-repo>
```

⚠️ _Chú ý:

Nếu đây là lần đầu bạn sử dụng Git, khi chạy xong lệnh `clone`, một thông báo pop-up sẽ hiện lên yêu cầu bạn đăng nhập để kết nối đến tài khoản Github của bạn, hãy chọn tài khoản đã được mời vào dự án và tiến hành đăng nhập. Khi đó Git sẽ nhận diện được bạn có quyền thao tác lên dự án hay không.

Sau khi chạy lệnh, bạn sẽ thấy Git tạo một thư mục ẩn `.git` trên folder này. Đây là file để Git quản lý, **không cần quan tâm đến nó**. Ngay sau đó, git sẽ kéo repo từ nguồn remote về máy bạn. Khi đó bạn sẽ thấy các file dự án từ Github sẽ được kéo về trên folder local của bạn.

B. Bắt đầu công việc: Tạo nhánh mới (Branching)

⚠️ _Chú ý:

Mọi thao tác lên dự án bằng Git đều phải được thực hiện trên chính folder các bạn đã khởi tạo Git / Clone dự án (tức phải có thư mục `.git`). Nếu bạn thao tác ở một thư mục khác, khi đẩy các file lên thì cấu trúc dự án sẽ bị đảo lộn và ngược lại ở phía local khi kéo về cũng vậy. Do đó luôn luôn ghi nhớ điều này khi kéo/đẩy code.

⚠️ _Quy tắc vàng:

TUYỆT ĐỐI KHÔNG code trực tiếp trên nhánh `main` hoặc `master`. Mỗi tính năng hoặc lỗi cần sửa cần phải nằm trên một nhánh riêng. Nhánh này cũng sẽ là nhánh được đẩy lên remote. Việc ghép code (Merge) vào nhánh chính (`main` hoặc `master`) là trách nhiệm của Team Lead.

- B1: Kiểm tra xem mình đang ở nhánh nào:

```
git branch
```

Khi đó, Git sẽ hiển thị các nhánh đang có trên local (`master` và một số nhánh của bạn)

- B2: Thao tác với các nhánh:

Quy tắc đặt tên:

Trên nhánh của bạn luôn phải đặt tên có tiền tố. Việc này giúp ta dễ dàng hơn trong việc truy vấn lại code sau này:

wip/	Works in progress (Đang làm). Chưa làm xong nhưng phải đẩy lên.
feat/	Feature. Một chức năng bạn thêm vào.
bug/	Bug. Sửa hoặc refactor code.
junk/	Code ở tình trạng thử nghiệm.
test/	Nhánh dành cho tester

Khi này, giả sử bạn chạy lệnh `git branch --list "feat/*"` để truy vấn lại các chức năng bạn đã từng làm, kết quả trả ra sẽ là:

```
feat/login
feat/products
feat/services
```

Vận dụng quy tắc trên, bạn thử tạo ra một nhánh mới bằng câu lệnh:

```
git checkout -b <tên-nhánh-của-bạn>
```

Ví dụ: `git checkout -b feat/login` hoặc `git checkout -b test/login`

Ở đây, `-b` cho Git biết rằng bạn muốn tạo một nhánh mới chưa từng xuất hiện, nếu bạn chỉ muốn chuyển từ `master` sang một nhánh hiện có, chỉ cần bỏ `-b` đi là xong:

```
git checkout <tên-nhánh-của-bạn>
```

C. Quy trình làm việc hàng ngày (Add - Commit - Pull - Push)

Đây là vòng lặp bạn sẽ sử dụng liên tục mỗi khi viết code xong một phần nào đó.

Bước 1: Kiểm tra trạng thái file (Bắt buộc làm thường xuyên)

Xem những file nào đã bị thay đổi, file nào mới được tạo.

```
git status
```

Khi này Git sẽ trả về toàn bộ trạng thái hiện tại trên local của bạn, có những file nào chưa được `stage`, những file nào chưa được `commit`, những file nào đang bị CONFLICT,...

Bước 2: Thêm file vào danh sách chuẩn bị lưu (Staging Area)

Khi bạn sửa code xong, bạn cần lưu file lại và thông báo với Git rằng bạn đang cho các file hiện tại vào một "giỏ hàng" (commit) để được lưu lại trong hệ thống.

- Cách 1: Thêm **tất cả** các file đã thay đổi (Trên đường dẫn folder hiện tại):

```
git add .
```

- Cách 2: Thêm **từng file** cụ thể (Khuyến dùng nếu có file không muốn đẩy lên):

```
git add <tên-file-kèm-đuôi-file>
```

Bước 3: Lưu lại thay đổi (Commit)

Gói gọn các thay đổi lại trong một "giỏ hàng" với một lời nhắn rõ ràng để team biết bạn vừa làm gì với các thay đổi đó:

```
git commit -m "Thêm tính năng đăng nhập bằng Google"
```

Đây là một quyết định rất chuẩn xác! Việc sử dụng mô hình **Feature Branch + Pull Request (PR)** là quy chuẩn của các dự án phần mềm chuyên nghiệp. Nó giúp bảo vệ nhánh chính

(`dev` / `main`) không bị lỗi và giúp Team Lead kiểm soát được chất lượng code (Code Review) trước khi gộp vào.

Bước 4: Đẩy nhánh làm việc cá nhân lên mạng (Push):

Tuyệt đối không đẩy trực tiếp code của bạn vào nhánh `dev` hoặc `master`. Hãy đẩy toàn bộ công việc lên nhánh riêng của bạn (nhánh bạn đã tạo ở phần B) trên server.

```
git push origin <tên-nhánh-của-bạn>
```

Ví dụ: `git push origin feat/login`

Bước 5: Tạo Yêu cầu gộp code (Pull Request / Merge Request) Đây là bước bạn "nộp bài" cho Team Lead duyệt.

- B1: Mở trang dự án trên GitHub/GitLab bằng trình duyệt.
- B2: Bạn sẽ thấy một thông báo màu xanh lá cây hoặc nút "**Compare & pull request**" (hoặc Create Merge Request) xuất hiện cạnh tên nhánh của bạn. Hãy bấm vào đó.
- B3: Điền tiêu đề và mô tả ngắn gọn những gì bạn đã làm trong nhánh này.
- B4: Bấm "**Create pull request**".

Bước 6: Chờ Team Lead duyệt (Review & Merge) Lúc này, công việc của bạn tạm xong. Team Lead sẽ vào đọc code của bạn:

- Nếu code tốt và không có xung đột (Conflict), **Team Lead** sẽ là người bấm nút **Merge** để gộp code của bạn vào nhánh chung của team.
- Nếu có đoạn cần sửa đổi hoặc xảy ra Conflict với code của người khác, Lead sẽ comment trực tiếp trên Pull Request. Bạn chỉ cần quay lại máy, sửa code, add-commit rồi gõ lại lệnh `git push origin <tên-nhánh-của-bạn>`. Pull Request trên mạng sẽ tự động cập nhật code mới của bạn.

Best Practices (Những lưu ý đặc biệt cho người mới)

1. **Đọc kỹ câu trả lời của `git status`**: Git rất thông minh. Khi bạn gõ `git status`, nó thường gợi ý luôn lệnh tiếp theo bạn cần gõ (ví dụ như cách để bỏ add một file). Hãy tập thói quen đọc thông báo của Git.
2. **Commit nhỏ và thường xuyên**: Đừng đợi code xong cả một trang web rồi mới commit. Hãy commit ngay khi xong một chức năng nhỏ (VD: "Tạo xong form UI", "Viết logic validate email"). Việc này giúp dễ tìm lỗi và rollback nếu lỡ tay làm hỏng code.
3. **Viết Commit Message có ý nghĩa**: Tránh những message vô thưởng vô phạt như `git commit -m "update"` hay `"fix bug"`. Hãy viết rõ: `"feat: Add Google login"` hoặc `"Refactor: Deleted fields in Products"`.
4. **Sử dụng `.gitignore`**: Luôn đảm bảo dự án có file `.gitignore` để tránh đẩy nhầm các file rác, file cấu hình cá nhân hoặc các thư mục nặng (như `node_modules`, `target`, `bin`,

`obj`) lên server.

5. Luôn cập nhật code mới nhất trước khi push: Hãy chạy lệnh `git pull origin master` (hoặc `dev`) để kéo code mới nhất từ nhánh chính về nhánh cá nhân của bạn. Nếu có Conflict, xử lý ngay trên Local, add & commit lại, rồi mới gõ `git push` lên nhánh riêng của bạn trên remote để tạo Merge Request.
6. **Nếu gặp Conflict (Xung đột code):** Đừng hoảng sợ! Đừng tự ý xóa code của người khác. Hãy báo ngay cho người quản lý nhánh hoặc người viết đoạn code đó để cùng nhau gỡ (Resolve Conflict). Chụp lại ảnh của code đang bị conflict được đánh dấu và gửi lên Zalo.